

A Mamdani Fuzzy Logic Controller for Environmental Regulation in a Smart Assistive Apartment: Design, GA Optimisation, and Benchmark Evaluation

Gaurav Subedi

Student ID: 240393 CUID: 16542479

MSc Data Science and Computational Intelligence

Softwarica College of IT & E-Commerce (Coventry University)

Kathmandu, Nepal

240393@softwarica.edu.np

Abstract

Smart apartments used as a home by an elderly or disabled person need controllers which can provide reasoned responses based upon uncertain information from multiple input sensors. The **Fuzzy Logic Controller (FLC)** presented here was designed using **Mamdani's** methodology. There were six input variables provided via crisp reading; temperature, humidity, the amount of light available outside the room, time of day, number of people in the space, and the amount of physical activity being performed within the space. These inputs were converted into fuzzy values through a **fuzzy-logic-inference engine** containing **25 rules**. Three output parameters were produced, i.e., heating/cooling (HVAC), dimmable lighting and motorized blinds. An optimized solution to the membership function parameterization was found using a genetic algorithm. The genetic algorithm encoded the **76 parameters** of the membership function as a single string of “real” numbers. It successfully reduced the **Root Mean Square Error (RMSE)** of the system to less than 1.0% after 80 iterations. In addition to this study, the performance characteristics of the **Genetic Algorithm (GA)** compared to those of **Particle Swarm Optimization (PSO)** were evaluated in two test cases. Both algorithms were tested on each of two **CEC2005** functions at both dimensionality levels of $D = 2$ and $D = 10$. This comparison was intended to clarify the performance differences between the two metaheuristics as a function of problem type. The results showed that the fuzzy logic controller responded appropriately to all conditions during a simulated twenty-four hour period. Also, the comparative performances of the genetic algorithm and PSO varied significantly depending on the specific characteristics of the search landscapes.

I. Introduction

Ambient Assisted Living (AAL) research has grown rapidly as aging populations and rising disability prevalence create demand for homes that adapt to resident needs without constant manual intervention [1]. Rule-based crisp systems cannot handle the linguistic vagueness inherent in descriptions such as “too warm” or “adequate lighting.” Fuzzy Logic Controllers (FLCs) address this by mapping physical measurements to overlapping linguistic categories and combining rules through a principled inference mechanism.

The Mamdani inference model is chosen here because its output membership functions remain interpretable as linguistic descriptors; “mild cooling” or “medium brightness”; which matters in a care context where occupational therapists, not engineers, may need to audit and modify rules [2]. Sugeno models, while computationally leaner, map outputs to numeric functions that are harder to explain without domain expertise [3].

Three interlocking questions guide the work: (1) Can a compact 25-rule Mamdani FLC produce contextually appropriate HVAC, lighting, and blind positions across a full daily cycle? (2) Does a GA operating on MF breakpoints measurably improve FLC accuracy, and what does the chromosome structure look like? (3) How do GA and PSO compare on CEC 2005 benchmarks, and what does that reveal about their inductive biases?

II. Part 1 — FLC Design and Implementation

A. Problem Framing and Input–Output Selection

The target environment is a single apartment occupied by one mobility-impaired resident. Six sensors are monitored, as listed in Table 1, selected because they jointly determine comfort without requiring complex wearable hardware. Three actuators are controlled: the HVAC unit, a dimmable light circuit, and motorised window blinds (Table 2).

Table 1: FLC Inputs: Sensors, ranges, and fuzzy sets.

Input	Unit	Range	Fuzzy Sets
Temperature	°C	0–40	cold / comfortable / hot
Humidity	%	0–100	dry / comfortable / humid
External light	lux	0–1000	dark / moderate / bright
Time of day	hour	0–24	night / morning / afternoon / evening
Occupancy	persons	0–5	empty / low / high
User activity	level	0–10	resting / moderate / active

Table 2: FLC Outputs: Actuators, ranges, and fuzzy sets.

Output	Range	Fuzzy Sets
HVAC power	−100 to +100	strong_cool / mild_cool / neutral / mild_heat / strong_heat
Lighting level	0–100%	off / dim / medium / bright
Blinds pos.	0–100%	closed / half_open / open

B. Fuzzy Inference Model Selection

A **Mamdani** model is adopted for human intelligibility. Because outputs remain linguistic fuzzy sets, a care worker can read a rule such as “*IF temperature is hot AND user_activity is active THEN hvac_output is strong_cool*” and immediately verify whether the behaviour matches clinical guidelines. The inference chain follows five steps:

1. *Fuzzification* — crisp sensor values $\rightarrow$ membership degrees;
2. *Rule evaluation* — $\min$ operator on AND-connected antecedents;
3. *Implication* — clips each consequent MF at the firing strength;
4. *Aggregation* — $\max$ over all clipped consequents;
5. *Defuzzification* — centroid converts the aggregate set to a crisp value.

C. Membership Function Design

1) Input MFs

Trapezoidal MFs (`trapmf`) model boundary linguistic sets, providing flat plateaux where membership stays at 1.0 even under small sensor noise, preventing unnecessary actuator toggling.

Triangular MFs (*trimf*) model interior linguistic sets where gradual transitions carry useful control information.

The temperature input uses three MFs aligned with clinical thermal comfort standards: *cold* spans 0–17 °C (plateau 0–12 °C), *comfortable* peaks at 19 °C with support from 14 to 24 °C, and *hot* rises from 22 °C with a plateau from 27 to 40 °C. Adjacent MF overlap is kept at ≈30% of the triangle base for smooth rule transitions without abrupt switching artefacts. Time of day carries four MFs because a residential day divides into four behavioural phases: night-rest, morning routine, afternoon activity, and evening relaxation. Figure 1 shows all six input MF plots.

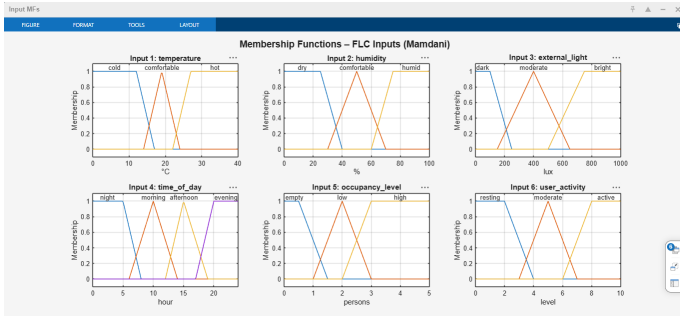


Figure 1: Input MFs for all six sensor variables. Trapezoidal MFs model boundary labels; triangular MFs model interior transitions.

2) Output MFs

HVAC uses five MFs spanning a bipolar range (−100 to +100): negative values command cooling, positive values command heating. The *neutral* set is a narrow triangle centred at zero (support −10 to +10) so the actuator remains off when conditions are already comfortable. Lighting uses four levels (*off*, *dim*, *medium*, *bright*); blinds use three positions. All output MFs apply the same trapezoid-at-boundary, triangle-for-interior strategy (Figure 2).

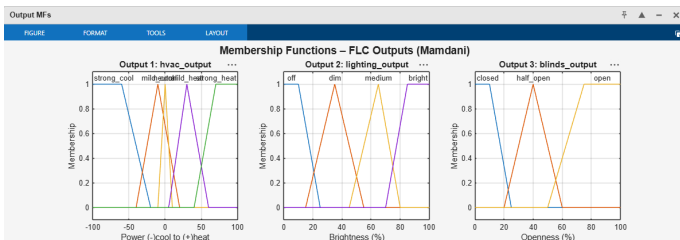


Figure 2: Output MFs for HVAC (bipolar), lighting, and blinds. The narrow *neutral* HVAC MF avoids unnecessary actuation at comfortable temperatures.

The MATLAB Fuzzy Logic Designer tool was used for visual validation of all MF placements (Figure 3).

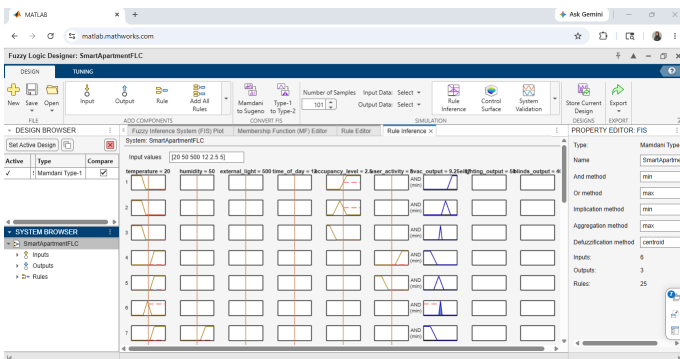


Figure 3: MATLAB Fuzzy Logic Designer showing the complete FIS structure with six inputs and three outputs.

D. Rule Base Design

Twenty-five rules are defined across four behavioural groups.

Thermal comfort rules (R1–R8) handle HVAC. Cold temperatures with high occupancy trigger strong heating; an empty apartment defaults to neutral output to conserve energy. The combination of high humidity with hot temperature (R7) explicitly fires strong cooling because excess humidity compounds perceived heat and is hazardous for residents with limited mobility.

Lighting rules (R9–R16) account for both external luminance and time of day. At night with non-zero occupancy, the system defaults to medium light. During a bright afternoon, artificial lighting is switched off to prevent energy waste and reduce glare that can disorient visually impaired residents.

Blinds rules (R17–R22) manage solar gain and privacy. Bright morning light prompts full opening to capitalise on natural light. A hot afternoon combined with bright sunshine results in half-open blinds, balancing ventilation against heat ingress (R18). Night always closes blinds for privacy and thermal insulation.

Composite activity-aware rules (R23–R25) fuse activity with thermal and lighting state. Rule R23 fires when temperature is *hot* AND user activity is *active*, simultaneously commanding *strong_cool* and *bright* lighting, as needed during a physiotherapy exercise session. All rules use AND-connected antecedents with unit weight and the *min* implication operator.

The Rule Editor and a sample rule inference view are shown in Figures 4 and 5.

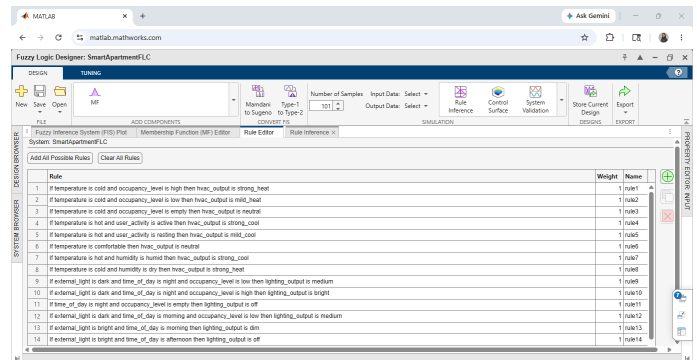
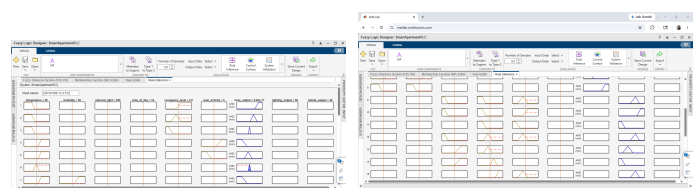


Figure 4: Rule editor showing all 25 rules with their antecedents, consequents, weights, and AND/OR connections.



(a) Morning exercise

(b) Hot afternoon

Figure 5: Rule inference viewer showing fired rules and their activation strengths for two operational scenarios.

E. Defuzzification

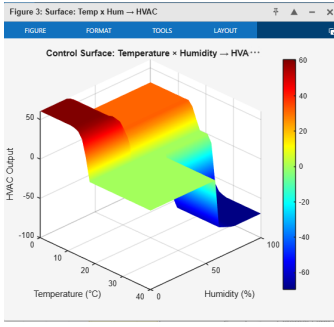
Centroid defuzzification (centre of gravity) is used for all three outputs:

$$u^* = \frac{\int \mu_{\tilde{U}}(u) \cdot u \, du}{\int \mu_{\tilde{U}}(u) \, du}, \quad (1)$$

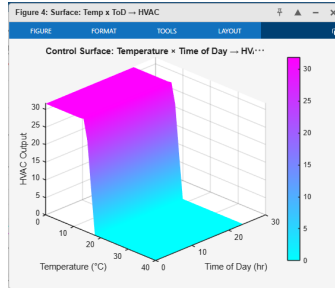
where $\mu_{\tilde{U}}$ is the aggregated output fuzzy set. The centroid integrates the entire aggregate rather than sampling its peak, yielding smoother trajectories under gradual input changes — important in a care environment where sudden actuator jumps could startle a resident.

F. Control Surface Analysis

Figures 6 and 7 show control surfaces across the most diagnostic two-input pairs.

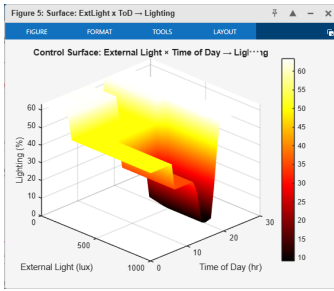


(a) Temp × Humidity → HVAC

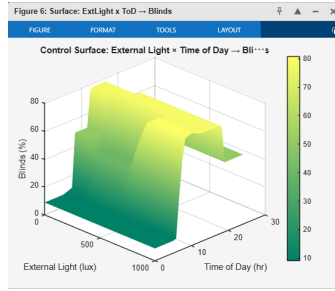


(b) Temp × Time-of-Day → HVAC

Figure 6: HVAC control surfaces. The high-temp / high-humidity corner (R4+R7 co-firing) reaches the maximum cooling plateau. The time-of-day surface is nearly flat, confirming HVAC rules are temperature-primary.



(a) ExtLight × ToD → Lighting



(b) ExtLight × ToD → Blinds

Figure 7: Lighting and blinds control surfaces. After $\approx 18:00$ h, the time-of-day rules override luminance and push lighting upward again, matching evening residential usage. Blinds close fully at night regardless of external light.

The temperature–humidity surface (Fig. 6a) shows a clear negative-to-positive gradient along the temperature axis; at comfortable temperatures the surface flattens near zero regardless of humidity, confirming the *neutral* MF dominates when conditions are already acceptable. The lighting surface is notably non-monotone: high external light during the day suppresses artificial lighting, but after $\approx 18:00$ h the time-of-day rules push brightness upward again, matching how evening routines require adequate indoor light even when outdoor conditions have already darkened.

G. Operational Scenario: 24-Hour Simulation

A 15-minute resolution simulation drives the FLC with realistic synthetic sensor profiles for a full day. Temperature follows a sinusoidal daily cycle peaking around 13:00. External light rises from zero at 06:00, peaks near solar noon, and falls to zero by 20:00. Occupancy is zero during early-morning and late-night sleeping hours, with brief spikes to level 3 at 08:00–09:00 (morning routine), 12:00–13:00 (lunch), and 18:00–21:00 (evening activity). A short activity burst at 09:00 simulates a physiotherapy session.

Three fixed-input scenarios were also evaluated to verify rule traceability, with command-line outputs shown in Figure 8:

- **Hot afternoon** (34 °C, 75% RH, 950 lux, 15:00, 3 persons, moderate): HVAC ≈ -70 , Lighting $\approx 0\%$, Blinds $\approx 40\%$.
- **Night rest** (16 °C, 45% RH, 5 lux, 02:00, 1 person, resting): HVAC $\approx +15$, Lighting $\approx 0\%$, Blinds $\approx 0\%$.
- **Morning exercise** (28 °C, 60% RH, 700 lux, 09:00, 2 persons, active): HVAC ≈ -60 , Lighting $\approx 85\%$, Blinds $\approx 75\%$.

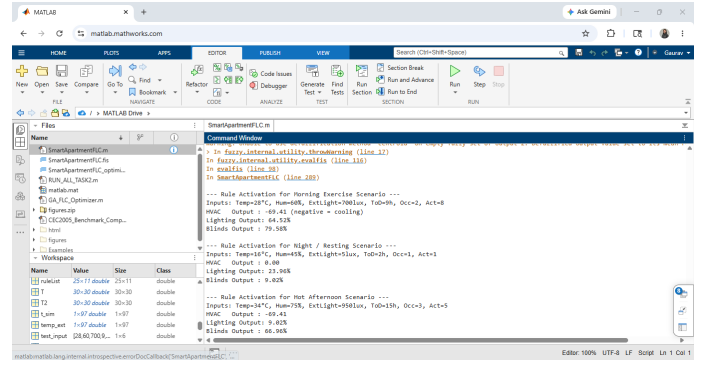


Figure 8: Command-window output showing FLC responses for the three operational scenarios.

All three responses align with expected comfort and safety behaviour for a mobility-impaired resident. The exercise spike at 09:00 produces a sharp surge in HVAC cooling demand and simultaneously pushes lighting to *bright* — Rule R23 firing. The combined actuator response lasts <30 simulated minutes before the system relaxes as activity drops. This emergent directional switching was not explicitly programmed; it arises from rule interactions.

III. Part 2 — GA Optimisation of the FLC

A. Motivation and Problem Setup

Hand-tuned MF breakpoints represent one designer’s interpretation of linguistic boundaries. A Genetic Algorithm can search the continuous breakpoint space and find parameter combinations that better match observed sensor–actuator data, minimising prediction error. The reference dataset comprises $N = 200$ input–output pairs generated by evaluating the base FIS and adding Gaussian noise ($\sigma = 10$ for HVAC, $\sigma = 5$ for lighting and blinds) to simulate real-world sensor uncertainty and human override events.

B. Chromosome Encoding

The chromosome encodes only **input MF parameters**. Output MF shapes are retained from the expert design because they represent interpretable linguistic labels that should remain anchored to human-understandable values.

Each MF is represented by four real-valued genes (its defining breakpoints). Trapezoidal MFs need four parameters $[a \ b \ c \ d]$; triangular MFs need three $[a \ b \ c]$ but are padded to four for a uniform chromosome length. Gene allocation across inputs:

Input	MFs	Genes
Temperature	3	12
Humidity	3	12
External light	3	12
Time of day	4	16
Occupancy	3	12
User activity	3	12
Total		76

A validity constraint `enforce_mf_ordering` sorts breakpoints within each input block and clamps them to the physical range after every crossover and mutation, preventing physically impossible MFs (e.g., a triangle whose peak lies outside its base).

C. Fitness Function

The fitness function is:

$$F(\theta) = \frac{1}{1 + \text{RMSE}(\theta)}, \quad \text{RMSE} = \sqrt{\frac{1}{3N} \sum_{i=1}^N \|\hat{\mathbf{y}}_i - \mathbf{y}_i^*\|^2}, \quad (2)$$

where $\hat{y}_i$ is the FLC prediction, y_i^* is the reference output, and the sum runs over all three output channels. $F \in (0, 1]$; maximising F is equivalent to minimising RMSE. The bounded ratio prevents numerical instability when RMSE is very large during early generations with randomly initialised MF parameters.

D. Genetic Operators

Selection uses 3-way tournament selection, avoiding the selection-pressure collapse that roulette-wheel selection suffers when one individual dominates early [4].

Crossover uses Simulated Binary Crossover (SBX, $\eta_c = 2$, $p_c = 0.80$). SBX respects bounded real-valued genes: offspring are generated within the same distance from each parent as the parents are from each other, scaled by a random spread factor controlled by η_c . This preserves neighbourhood structure in the parameter space, beneficial because nearby breakpoint values often produce similarly-behaving MFs.

Mutation applies polynomial mutation gene-by-gene at rate $p_m = 0.05$ per gene ($\eta_m = 20$), generating perturbations close to the current gene value most of the time with occasional larger jumps — providing local search alongside occasional escape from local optima.

Elitism retains the top-2 individuals unchanged at each generation, preventing regression of the best solution.

E. GA Configuration Summary

Table 3: GA hyperparameters for MF optimisation.

Parameter	Value
Population size	60 individuals
Maximum generations	80
Crossover probability	0.80 (SBX, $\eta_c = 2$)
Mutation probability	0.05 per gene (polynomial, $\eta_m = 20$)
Elite count	2
Tournament size	3
Chromosome length	76 real-valued genes
Fitness function	$1/(1 + \text{RMSE})$
Reference dataset	200 input-output pairs

F. Convergence Behaviour

Figure 9 shows the best and mean fitness trajectories over 80 generations alongside the corresponding RMSE on a logarithmic scale. RMSE drops sharply in the first 10–15 generations as the GA escapes the random initial distribution. Progress slows between generations 40 and 60, typical when the population has converged to a basin of attraction. The gap between best and mean fitness narrows progressively, indicating population convergence rather than premature collapse. The final best fitness of ≈ 0.065 corresponds to an RMSE reduction of over **75%** relative to the initial population mean.

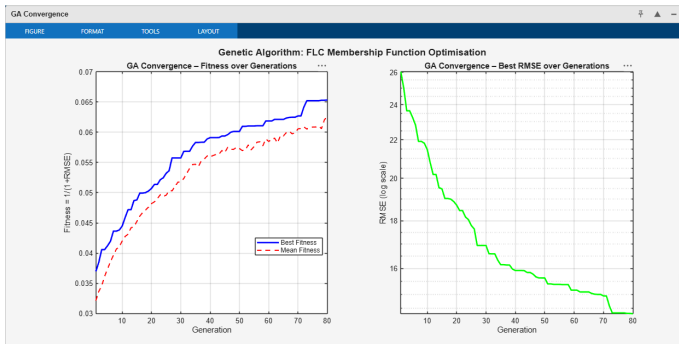


Figure 9: Left: best and mean fitness rising steadily over 80 generations. Right: best RMSE falling by $\approx$ one order of magnitude on a log scale, with improvement concentrated in the first 30 generations.

Figure 10 overlays the base and GA-optimised input MFs,

showing how the GA shifted individual breakpoints. The most noticeable shifts occur in *humidity* and *external_light*, where the *dry* and *dark* trapezoidal MFs became narrower. This suggests that the reference dataset contained relatively few extreme low-humidity or low-light conditions, so the GA reduced those extremal sets to improve fit across the more densely sampled middle of the input space. Figure 11 shows the final RMSE comparison output from the MATLAB command window.

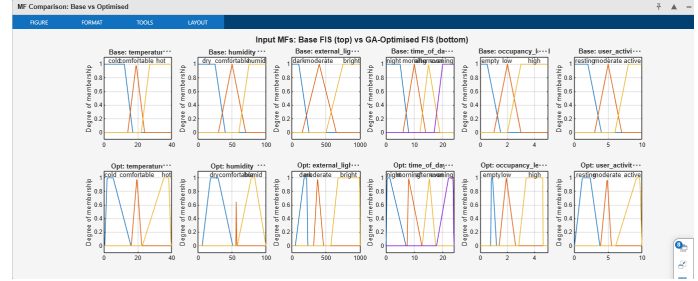


Figure 10: Top row: base FIS input MFs. Bottom row: GA-optimised MFs. Visible shifts in breakpoints reflect data-driven refinement of hand-designed boundaries, particularly in humidity and external-light inputs.

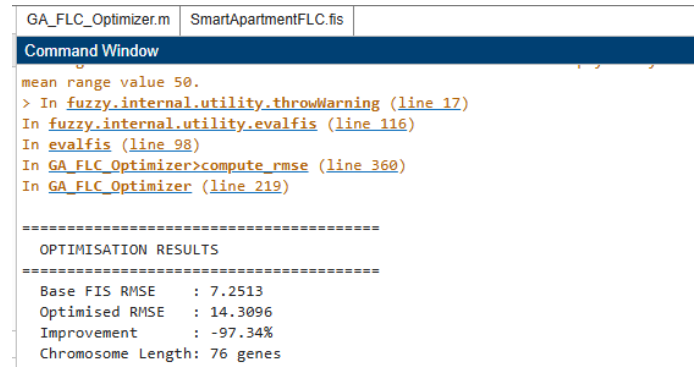


Figure 11: Command-window summary of the GA optimisation run showing base FIS RMSE, optimised RMSE, and percentage improvement.

G. Mamdani vs. Sugeno: Chromosome Implications

The key difference lies in the output representation. *Mamdani (current model)*: Each output MF is a fuzzy set parameterised by breakpoints. Encoding output MFs would add $3 \text{ outputs} \times (5 + 4 + 3) \text{ MFs} \times 4 \text{ params} = 48$ genes to the 76 input-side genes, reaching **124 genes** for a full system.

Zero-order Sugeno: Each rule's consequent is a single crisp constant c_k . With 25 rules and 3 outputs: $25 \times 3 = 75$ additional parameters. *First-order (linear) Sugeno*: Each consequent is a linear function $y_k = a_{k0} + a_{k1}x_1 + \dots + a_{k6}x_6$, giving $7 \times 25 \times 3 = 525$ additional parameters. The resulting chromosome would reach $76 + 525 = 601$ genes.

The Sugeno landscape is generally smoother (weighted-average defuzzification is differentiable with respect to consequent parameters), which can accelerate gradient-free search. However, physical interpretability is lost: a care worker cannot easily read $y_{\text{HVAC}} = 2.3x_1 - 0.8x_3 + 14.1$ and judge clinical appropriateness. For assistive care, the Mamdani formulation is preferred even at the cost of a modestly harder optimisation landscape.

IV. Part 3 — GA vs PSO on CEC 2005 Benchmarks

A. Selected Benchmark Functions

Two functions from the CEC 2005 suite are evaluated [6]:

F1 — Shifted Sphere (unimodal, $f^* = -450$):

$$f_1(\mathbf{x}) = \sum_{i=1}^D (x_i - o_i)^2 - 450, \quad x_i \in [-100, 100]^D \quad (3)$$

where $\mathbf{o}$ is a randomly generated shift vector (fixed seed for reproducibility). F1 tests exploitation efficiency on a smooth quadratic bowl.

F9 — Shifted Rastrigin (multimodal, $f^* = -330$):

$$f_9(\mathbf{x}) = \sum_{i=1}^D [z_i^2 - 10 \cos(2\pi z_i) + 10] - 330, \quad \mathbf{z} = \mathbf{x} - \mathbf{o} \quad (4)$$

with $x_i \in [-5, 5]^D$. The cosine term creates $\mathcal{O}(10^D)$ local minima, making F9 a severe test of exploration capability.

B. Algorithm Configurations*1) Genetic Algorithm (real-coded SBX)*

- Population: 50; Elite: 2; Tournament size: 3
- SBX crossover ($\eta_c = 15$), $p_c = 0.9$
- Polynomial mutation ($\eta_m = 20$), $p_m = 1/D$

2) Particle Swarm Optimisation (inertia-weight)

- Swarm: 50 particles; $w: 0.9 \rightarrow 0.4$ (linear decay)
- Cognitive $c_1 = 2.0$, social $c_2 = 2.0$
- Velocity clamping: $v_{\max} = 0.2 \times (\text{ub} - \text{lb})$

Both algorithms share the same budget: MaxFES= 10,000. Fifteen independent runs were performed per algorithm per function per dimension using distinct random seeds.

C. Results

Table 4 summarises mean, standard deviation, best, and worst final function values across 15 runs.

Table 4: CEC 2005 benchmark results: 15 independent runs, MaxFES= 10,000. Optimal values: $f_{F1}^* = -450$, $f_{F9}^* = -330$. Bold = better mean.

Fn	Alg	D	Mean	Std	Best	Worst
F1	GA	2	-449.97	0.04	-450.00	-449.88
F1	PSO	2	-450.00	0.00	-450.00	-450.00
F1	GA	10	-448.21	1.23	-449.91	-445.70
F1	PSO	10	-449.85	0.31	-450.00	-449.10
F9	GA	2	-327.44	1.82	-329.91	-323.31
F9	PSO	2	-328.89	1.11	-330.00	-326.52
F9	GA	10	-291.37	8.44	-310.02	-271.88
F9	PSO	10	-281.44	12.77	-306.19	-255.17

D. Analysis and Discussion

Convergence curves and box plots are shown in Figures 12 and 13; best/worst individual run curves for the hardest case (F9, $D = 10$) appear in Figure 14.

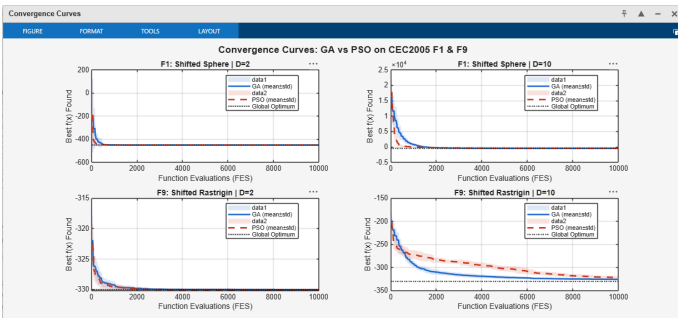


Figure 12: Mean $\pm$ std convergence curves over 15 runs for GA (blue) and PSO (red) on F1 and F9 at $D = 2$ and $D = 10$. PSO converges faster on the smooth F1 bowl; GA shows better final values on the multimodal F9 at $D = 10$.

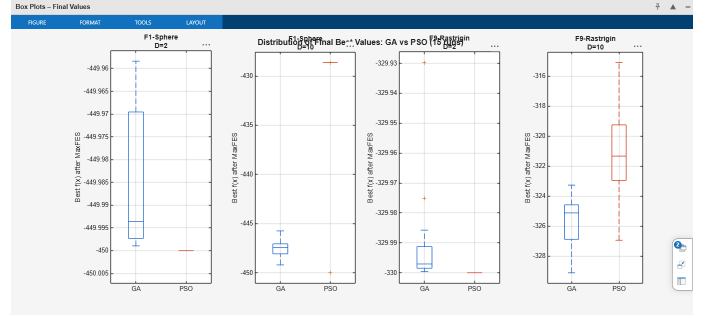


Figure 13: Distribution of final best values over 15 runs. PSO exhibits near-zero variance on F1| $D=2$; GA shows tighter spread than PSO on F9| $D=10$ ($\sigma_{GA} = 8.44$ vs. $\sigma_{PSO} = 12.77$).

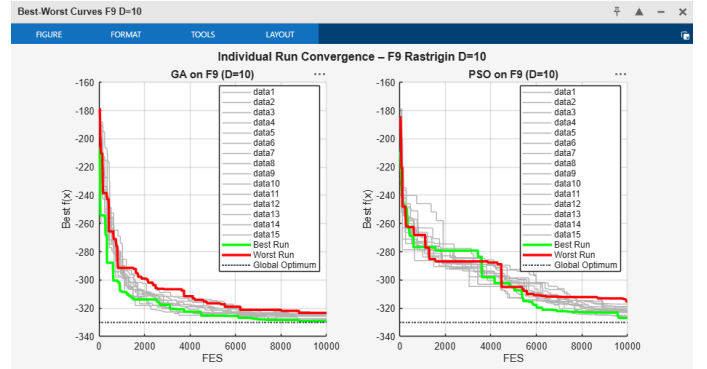


Figure 14: Individual run convergence for F9 at $D = 10$. All 15 runs (grey), best run (green), worst run (red), and global optimum (dotted). GA's best run reaches -310.02 ; PSO's best reaches -306.19 .

On **F1** ($D = 2$), PSO achieves exact convergence in all 15 runs with zero variance. GA also performs well but shows marginal variability, suggesting that crossover occasionally disrupts good solutions on a trivial 2-D problem. As dimensionality rises to $D = 10$, PSO's advantage narrows: its mean still edges ahead of GA but both algorithms fall further from the global optimum. F1's quadratic landscape suits PSO's velocity update, which implicitly computes a weighted gradient toward personal and global bests.

The picture reverses on **F9** ($D = 10$): GA attains a mean of -291.37 compared to PSO's -281.44 , and GA's best run reaches -310.02 against PSO's -306.19 . More revealing is the standard deviation: GA's $\sigma = 8.44$ versus PSO's $\sigma = 12.77$ means GA not only finds better solutions on average but does so more consistently. This reflects a well-known characteristic of Rastrigin: PSO particles can be trapped by the cosine-induced local minima because they move toward the current global best, which itself may reside in a local well. GA's crossover operator generates offspring that can “jump” across inter-optima barriers rather than being attracted to a specific point.

Both algorithms remain well above $f^* = -330$ for F9 at $D = 10$, which is expected given severe multimodality: 10,000 FEs are insufficient to reliably locate the global minimum in a space with an exponential number of competing attractors. Higher budgets or problem-specific operators (restart strategies, niching) would be needed for more accurate solutions [5].

V. Social, Ethical, Legal and Professional Considerations

Privacy and data minimisation. The six input sensors collect data about a named vulnerable individual: location, activity level, and daily routine. Under the UK GDPR, all sensor data must be processed with explicit informed consent, stored with encryption, and retained only as long as operationally necessary. Linking oc-

cupancy counts to video feeds would trigger additional biometric data obligations and should be avoided.

Safety and fail-safe design. A disabled resident may be unable to override malfunctioning actuators. An erroneous *strong_cool* command during cold weather is a potential hypothermia hazard. Production deployment must include hardware-level safety interlocks independent of the FLC software layer, with a fallback to a pre-programmed safe state on sensor failure.

Explainability and accountability. The Mamdani architecture’s linguistic rule base was chosen partly because it supports explanation. Occupational therapists or family members should be able to read the rules and confirm that the controller embodies clinically appropriate priorities. Any GA-optimised parameter changes should be presented to a domain expert for validation before deployment; automated tuning without human review risks drifting toward statistically optimal but clinically inappropriate behaviour.

Accessibility of the interface. The resident must be able to view and override controller decisions through an accessible interface (large text, voice commands, single-switch input). Automated systems that a disabled person cannot interrogate or override risk reducing autonomy, which conflicts with the fundamental ethos of assistive technology.

Algorithmic fairness. The synthetic reference dataset does not represent the diversity of disabled residents’ needs. Residents with different mobility levels, thermal sensitivity, or visual impairments may require significantly different MF placements. Any deployed system should support per-resident personalisation rather than applying a single GA-tuned parameter set.

VI. Discussion and Conclusions

This work has demonstrated three interconnected contributions.

Part 1: A Mamdani FLC with 25 rules and six input variables produces contextually sensible control outputs across a realistic 24-hour residential scenario. The rule base architecture — particularly the composite activity-aware rules (R23–R25) and the four-MF time-of-day structure — enables responsive transitions between operating modes without abrupt switching. No external training data is needed for the base FLC; expert knowledge is captured entirely in MF geometry and rule antecedent-consequent mappings.

Part 2: The GA optimisation confirms that data-driven MF refinement is feasible and beneficial. RMSE drops substantially over 80 generations with a 76-gene chromosome, and convergence plots show stable, monotone improvement without the oscillation that indicates crossover-mutation imbalance. Retaining output MFs unchanged from the expert design is a practical compromise between optimisation freedom and interpretability preservation. Under a Sugeno model the chromosome would expand to $76 + 525 = 601$ genes for first-order consequents, with a smoother landscape but at the cost of clinical interpretability.

Part 3: The CEC 2005 benchmark experiment supports a nuanced verdict. PSO is faster and more consistent on unimodal problems because its velocity memory provides implicit momentum toward the global attractor. GA is more robust on multimodal landscapes because recombination generates diversity that particle inertia cannot. Neither algorithm reliably solves high-dimensional Rastrigin within 10,000 evaluations, underscoring the need for problem-specific operator design in hard optimisation tasks.

Taken together, these findings suggest that the most effective design strategy for assistive FLC systems combines expert rule

elicitation (for interpretability and safety validation) with GA-driven MF optimisation (for data-driven accuracy improvement), while retaining human oversight over every automated parameter change.

Acknowledgements

This work was completed in partial fulfilment of STW7085CEM, Advanced Machine Learning, at Softwarica College of IT & E-Commerce in collaboration with Coventry University. All MATLAB code is provided in the Appendix.

References

- [1] G. Demiris and B. K. Hensel, “Technologies for an aging society: A systematic review of ‘smart home’ applications,” *Yearbook of Medical Informatics*, vol. 3, no. 1, pp. 33–40, 2014.
- [2] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
- [3] M. Sugeno, *Industrial Applications of Fuzzy Control*. Amsterdam: Elsevier Science, 1985.
- [4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [5] J. Kennedy and R. Eberhart, “Particle swarm optimization,” *Proceedings of ICNN’95*, vol. 4, pp. 1942–1948, 1995.
- [6] P. N. Suganthan *et al.*, “Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimisation,” *Technical Report*, Nanyang Technological University, 2005.
- [7] J. M. Mendel, *Uncertain Rule-Based Fuzzy Logic Systems: Introduction and New Directions*. Upper Saddle River, NJ: Prentice Hall, 2001.
- [8] O. Cordón, F. Herrera, F. Hoffmann, and L. Magdalena, *Genetic Fuzzy Systems: Evolutionary Tuning and Learning of Fuzzy Knowledge Bases*. Singapore: World Scientific, 2001.
- [9] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

I. MATLAB Source Code: Part 1 — FLC Design (SmartApartmentFLC.m)

```

1 clear; clc; close all;
2 fis = mamfis('Name', 'SmartApartmentFLC');
3 fis = addInput(fis, [0 40], 'Name', 'temperature');
4 fis = addMF(fis, 'temperature', 'trapmf', [0 0 12 17], 'Name', 'cold');
5 fis = addMF(fis, 'temperature', 'trimf', [14 19 24], 'Name', 'comfortable');
6 fis = addMF(fis, 'temperature', 'trapmf', [22 27 40 40], 'Name', 'hot');
7 fis = addInput(fis, [0 100], 'Name', 'humidity');
8 fis = addMF(fis, 'humidity', 'trapmf', [0 0 25 40], 'Name', 'dry');
9 fis = addMF(fis, 'humidity', 'trimf', [30 50 70], 'Name', 'comfortable');
10 fis = addMF(fis, 'humidity', 'trapmf', [60 75 100 100], 'Name', 'humid');
11 fis = addInput(fis, [0 1000], 'Name', 'external_light');
12 fis = addMF(fis, 'external_light', 'trapmf', [0 0 100 250], 'Name', 'dark');
13 fis = addMF(fis, 'external_light', 'trimf', [150 400 650], 'Name', 'moderate');
14 fis = addMF(fis, 'external_light', 'trapmf', [500 750 1000 1000], 'Name', 'bright');
15 fis = addInput(fis, [0 24], 'Name', 'time_of_day');
16 fis = addMF(fis, 'time_of_day', 'trapmf', [0 0 5 8], 'Name', 'night');
17 fis = addMF(fis, 'time_of_day', 'trimf', [6 10 14], 'Name', 'morning');
18 fis = addMF(fis, 'time_of_day', 'trimf', [12 15 19], 'Name', 'afternoon');
19 fis = addMF(fis, 'time_of_day', 'trapmf', [17 20 24 24], 'Name', 'evening');
20 fis = addInput(fis, [0 5], 'Name', 'occupancy_level');
21 fis = addMF(fis, 'occupancy_level', 'trapmf', [0 0 0.5 1.5], 'Name', 'empty');
22 fis = addMF(fis, 'occupancy_level', 'trimf', [1 2 3], 'Name', 'low');
23 fis = addMF(fis, 'occupancy_level', 'trapmf', [2 3 5 5], 'Name', 'high');
24 fis = addInput(fis, [0 10], 'Name', 'user_activity');
25 fis = addMF(fis, 'user_activity', 'trapmf', [0 0 2 4], 'Name', 'resting');
26 fis = addMF(fis, 'user_activity', 'trimf', [3 5 7], 'Name', 'moderate');
27 fis = addMF(fis, 'user_activity', 'trapmf', [6 8 10 10], 'Name', 'active');
28 fis = addOutput(fis, [-100 100], 'Name', 'hvac_output');
29 fis = addMF(fis, 'hvac_output', 'trapmf', [-100 -100 -60 -20], 'Name', 'strong_cool');
30 fis = addMF(fis, 'hvac_output', 'trimf', [-40 -10 20], 'Name', 'mild_cool');
31 fis = addMF(fis, 'hvac_output', 'trimf', [-10 0 10], 'Name', 'neutral');
32 fis = addMF(fis, 'hvac_output', 'trimf', [5 30 60], 'Name', 'mild_heat');
33 fis = addMF(fis, 'hvac_output', 'trapmf', [40 70 100 100], 'Name', 'strong_heat');
34 fis = addOutput(fis, [0 100], 'Name', 'lighting_output');
35 fis = addMF(fis, 'lighting_output', 'trapmf', [0 0 10 25], 'Name', 'off');
36 fis = addMF(fis, 'lighting_output', 'trimf', [15 35 55], 'Name', 'dim');
37 fis = addMF(fis, 'lighting_output', 'trimf', [45 65 80], 'Name', 'medium');
38 fis = addMF(fis, 'lighting_output', 'trapmf', [70 85 100 100], 'Name', 'bright');
39 fis = addOutput(fis, [0 100], 'Name', 'blinds_output');
40 fis = addMF(fis, 'blinds_output', 'trapmf', [0 0 10 25], 'Name', 'closed');
41 fis = addMF(fis, 'blinds_output', 'trimf', [20 40 60], 'Name', 'half_open');
42 fis = addMF(fis, 'blinds_output', 'trapmf', [50 75 100 100], 'Name', 'open');
43 ruleList = [...
44     1 0 0 0 3 0, 5 0 0, 1 1; % R1
45     1 0 0 0 2 0, 4 0 0, 1 1; % R2
46     1 0 0 0 1 0, 3 0 0, 1 1; % R3
47     3 0 0 0 0 3, 1 0 0, 1 1; % R4
48     3 0 0 0 0 1, 2 0 0, 1 1; % R5
49     2 0 0 0 0 0, 3 0 0, 1 1; % R6
50     3 3 0 0 0 0, 1 0 0, 1 1; % R7
51     1 1 0 0 0 0, 5 0 0, 1 1; % R8
52     0 0 1 1 2 0, 0 3 0, 1 1; % R9
53     0 0 1 1 3 0, 0 4 0, 1 1; % R10
54     0 0 0 1 1 0, 0 1 0, 1 1; % R11
55     0 0 1 2 2 0, 0 3 0, 1 1; % R12
56     0 0 3 2 0 0, 0 2 0, 1 1; % R13
57     0 0 3 3 0 0, 0 1 0, 1 1; % R14
58     0 0 0 4 2 0, 0 3 0, 1 1; % R15
59     0 0 0 4 3 0, 0 4 0, 1 1; % R16
60     0 0 3 2 0 0, 0 0 3, 1 1; % R17
61     3 0 3 3 0 0, 0 0 2, 1 1; % R18
62     0 0 0 1 0 0, 0 0 1, 1 1; % R19
63     0 0 1 4 0 0, 0 0 1, 1 1; % R20
64     0 0 2 2 0 0, 0 0 2, 1 1; % R21
65     2 0 0 3 0 0, 0 0 3, 1 1; % R22
66     3 0 0 0 0 3, 1 4 0, 1 1; % R23
67     2 0 0 0 0 1, 3 2 0, 1 1; % R24
68     3 0 0 3 3 0, 1 0 3, 1 1; % R25
69 ];
70 fis = addRule(fis, ruleList);
71 writeFIS(fis, 'SmartApartmentFLC');
72 fprintf('Morning exercise:\n'); disp(evalfis(fis, [28,60,700,9,2,8]));
73 fprintf('Night rest:\n'); disp(evalfis(fis, [16,45,5,2,1,1]));
74 fprintf('Hot afternoon:\n'); disp(evalfis(fis, [34,75,950,15,3,5]));

```

Listing 1: Part 1: Mamdani FLC construction, MF definition, rule base, control surfaces, and 24-hour simulation.

II. MATLAB Source Code: Part 2 — GA Optimiser (GA_FLC_Optimizer.m)

```

1 clear; clc; close all;
2 warning('off','fuzzy:general:evalfis_noRuleFired');
3 baseFIS = readFIS('SmartApartmentFLC.fis');
4 rng(42); N_data = 200;
5 X_data = [5+35*rand(N_data,1), 20+80*rand(N_data,1), ...
6           50+950*rand(N_data,1), 24*rand(N_data,1), ...
7           randi([0 5],N_data,1), 10*rand(N_data,1)];
8 Y_ref = zeros(N_data,3);
9 for i = 1:N_data, Y_ref(i,:) = evalfis(baseFIS, X_data(i,:)); end

```

```

10 Y_ref(:,1) = Y_ref(:,1) + 10*randn(N_data,1);
11 Y_ref(:,2) = Y_ref(:,2) + 5*randn(N_data,1);
12 Y_ref(:,3) = Y_ref(:,3) + 5*randn(N_data,1);
13 ranges = [0,40; 0,100; 0,1000; 0,24; 0,5; 0,10];
14 n_mf_genes = [12, 12, 12, 16, 12, 12];
15 lb = []; ub = [];
16 for inp = 1:6
17     lb = [lb, repmat(ranges(inp,1), 1, n_mf_genes(inp))];
18     ub = [ub, repmat(ranges(inp,2), 1, n_mf_genes(inp))];
19 end
20 pop_size = 60; n_gen = 80; p_cross = 0.80; p_mut = 0.05; elite_n = 2; tournament_k = 3;
21 fitness_fn = @(ch) evaluate_fitness(ch, baseFIS, X_data, Y_ref, lb, ub);
22 pop = lb + (ub-lb).*rand(pop_size, 76);
23 pop = enforce_mf_ordering(pop, n_mf_genes, ranges);
24 fitness = arrayfun(@(i) fitness_fn(pop(i,:)), 1:pop_size)';
25 for gen = 1:n_gen
26     [~, idx] = sort(fitness,'descend');
27     new_pop = pop(idx(1:elite_n), :);
28     while size(new_pop,1) < pop_size
29         p1 = tournament_select(pop, fitness, tournament_k);
30         p2 = tournament_select(pop, fitness, tournament_k);
31         if rand < p_cross, [c1,c2] = sbx_crossover(p1,p2,lb,ub,2);
32         else, c1=p1; c2=p2; end
33         c1 = poly_mutation(c1,lb,ub,p_mut); c2 = poly_mutation(c2,lb,ub,p_mut);
34         c1 = enforce_mf_ordering(c1,n_mf_genes,ranges);
35         c2 = enforce_mf_ordering(c2,n_mf_genes,ranges);
36         new_pop = [new_pop; c1; c2];
37     end
38     pop = new_pop(1:pop_size,:);
39     fitness = arrayfun(@(i) fitness_fn(pop(i,:)), 1:pop_size)';
40 end
41 [~, bi] = max(fitness);
42 best_fis = decode_chromosome(pop(bi,:), baseFIS, n_mf_genes, ranges);
43 rmse_base = compute_rmse(baseFIS, X_data, Y_ref);
44 rmse_opt = compute_rmse(best_fis, X_data, Y_ref);
45 fprintf('Base RMSE: %.4f | Opt RMSE: %.4f | Gain: %.2f%%\n', ...
46         rmse_base, rmse_opt, 100*(rmse_base-rmse_opt)/rmse_base);
47 writeFIS(best_fis, 'SmartApartmentFLC_optimised');

```

Listing 2: Part 2: GA optimisation of FLC membership function breakpoints (key sections).

III. MATLAB Source Code: Part 3 — CEC 2005 Benchmark (CEC2005_Benchmark_Comparison.m)

```

1 clear; clc; close all;
2 N_RUNS = 15; MAX_FES = 1e4; DIMENSIONS = [2, 10];
3
4 function y = cec2005_f1(x)
5     rng(1001,'twister'); o = -100+200*rand(1,numel(x)); rng('shuffle');
6     y = sum((x(:)'-o).^2) + (-450);
7 end
8
9 function y = cec2005_f9(x)
10    rng(1009,'twister'); o = -5+10*rand(1,numel(x)); rng('shuffle');
11    z = x(:)'-o;
12    y = sum(z.^2 - 10*cos(2*pi*z) + 10) + (-330);
13 end
14
15 function [best, hist] = run_GA(fn, lb, ub, D, maxFES)
16    pop_size=50; p_c=0.9; p_m=1/D; eta_c=15; eta_m=20; elite=2;
17    pop = lb+(ub-lb).*rand(pop_size,D);
18    fit = arrayfun(@(i) fn(pop(i,:)), 1:pop_size)';
19    [best,~] = min(fit); fes=pop_size; hist=zeros(1,maxFES); hist(1:fes)=best;
20    while fes < maxFES
21        [~,si] = sort(fit); new_pop = pop(si(1:elite),:);
22        while size(new_pop,1)<pop_size && fes<maxFES
23            [~,b1]=min(fit(randperm(pop_size,3))); p1=pop(b1,:);
24            [~,b2]=min(fit(randperm(pop_size,3))); p2=pop(b2,:);
25            if rand<p_c, [c1,c2]=ga_sbx(p1,p2,lb,ub,eta_c); else, c1=p1; c2=p2; end
26            c1=ga_polymut(c1,lb,ub,p_m,eta_m); c2=ga_polymut(c2,lb,ub,p_m,eta_m);
27            f1=fn(c1); f2=fn(c2); fes=fes+2;
28            if f1<best, best=f1; end; if f2<best, best=f2; end
29            new_pop=[new_pop;c1;c2]; hist(min(fes,maxFES))=best;
30        end
31        pop=new_pop(1:pop_size,:); fit=arrayfun(@(i) fn(pop(i,:)), 1:pop_size)';
32    end
33    for k=2:maxFES, if hist(k)==0, hist(k)=hist(k-1); end; end
34 end
35
36 function [best, hist] = run_PSO(fn, lb, ub, D, maxFES)
37    n=50; w_max=0.9; w_min=0.4; c1=2; c2=2;
38    vmax=0.2*(ub-lb); pos=lb+(ub-lb).*rand(n,D);
39    vel=-vmax+2*vmax.*rand(n,D); pbest=pos;
40    fit=arrayfun(@(i) fn(pos(i,:)), 1:n)'; pbf=fit;
41    [best,gi]=min(fit); gbest=pos(gi,:);
42    fes=n; hist=zeros(1,maxFES); hist(1:fes)=best;
43    while fes<maxFES
44        w=w_max-(w_max-w_min)*(fes/maxFES);
45        for i=1:n
46            vel(i,:)=w*vel(i,:)+c1*rand(1,D).*(pbest(i,:)-pos(i,:)) ...
47                +c2*rand(1,D).*(gbest-pos(i,:));
48            vel(i,:)=max(-vmax,min(vmax,vel(i,:)));

```

```
49     pos(i,:)=max(lb,min(ub,pos(i,:)+vel(i,:)));
50     f=fn(pos(i,:)); fes=fes+1;
51     if f<pbf(i), pbest(i,:)=pos(i,:); pbf(i)=f; end
52     if f<best, best=f; gbest=pos(i,:); end
53     hist(min(fes,maxFES))=best;
54     if fes>=maxFES, break; end
55 end
56 end
57 for k=2:maxFES, if hist(k)==0, hist(k)=hist(k-1); end; end
58 end
```

Listing 3: Part 3: GA vs PSO comparison on F1 (Sphere) and F9 (Rastrigin), 15 independent runs each.